

MT5 Trading Bots - Complete Source

Repository: github.com/Miten001/earn

Files included:

- requirements.txt - Python dependencies
- smc_ict_bot.py - SMC/ICT strategy bot (recommended)
- mt5_final_bot.py - Multi-Timeframe trend following bot
- mt5_bot.py - Scalping + Triangular arbitrage bot
- config.py - Shared config for v1 bot

Setup steps:

1. Install MT5 terminal on Windows and login (broker demo recommended).
2. pip install MetaTrader5 pandas numpy
3. Edit MT5_LOGIN / MT5_PASSWORD / MT5_SERVER at top of any bot file.
4. Run: python <botfile>.py

Path in repo: /smc_ict_bot.py

[illegible]

```

# Killzones (UTC). Only forex trades in killzone; crypto trades 24/7.
LONDON_KZ = (7, 10)      # 07:00 - 10:00 UTC
NEWYORK_KZ = (12, 15)    # 12:00 - 15:00 UTC

# Structure / pivot detection
PIVOT_LOOKBACK_H1 = 5    # 5-bar fractal on H1 (slower, cleaner pivots)
PIVOT_LOOKBACK_M15 = 3   # 3-bar fractal on M15
SWEEP_LOOKBACK_M15 = 30  # bars to look back for stop-hunt level
ATR_PERIOD = 14
SL_BUFFER_ATR = 0.3      # extra distance below sweep for SL

# Entry params
PENDING_ORDER_EXPIRY_HOURS = 4 # cancel limit if not filled in 4h
MIN_RR_TO_TARGET = 1.5        # skip setup if next pool < 1.5R away

# Trailing (same as v2 final bot)
BREAKEVEN_AT_R = 1.0
LOCK_HALF_AT_R = 1.5
TRAIL_START_R = 2.0
ATR_TRAIL_MULT = 1.5

# Spread caps
MAX_SPREAD_FX = 25
MAX_SPREAD_CRYPTO = 5000

# Loop
SCAN_INTERVAL_SEC = 10      # SMC setups are slower; 10s is plenty
RUN_MINUTES = 0             # 0 = forever
STATE_FILE = "smc_bot_state.json"

# =====

# ----- Indicators -----
def atr(df: pd.DataFrame, p: int = 14) -> pd.Series:
    h, l, c = df["high"], df["low"], df["close"]
    tr = pd.concat([h - l, (h - c.shift()).abs(), (l - c.shift()).abs()], axis=1).max(axis=1)
    return tr.ewm(alpha=1/p, adjust=False).mean()

# ----- MT5 helpers -----
TF = {"M5": mt5.TIMEFRAME_M5, "M15": mt5.TIMEFRAME_M15,
      "H1": mt5.TIMEFRAME_H1, "H4": mt5.TIMEFRAME_H4}

def init_mt5() -> bool:
    kw = {"login": MT5_LOGIN, "password": MT5_PASSWORD, "server": MT5_SERVER}
    if MT5_PATH:
        kw["path"] = MT5_PATH
    if not mt5.initialize(**kw):
        print(f"[!] MT5 init failed: {mt5.last_error()}")
        return False
    acc = mt5.account_info()
    if acc is None:
        print(f"[!] Login failed: {mt5.last_error()}")
        return False
    print(f"[+] Logged in: {acc.login} | Bal: {acc.balance:.2f} {acc.currency} | "
          f"Server: {acc.server}")
    return True

def find_symbol(base: str) -> Optional[str]:
    for cand in [base, base + "m", base + "r", base + "_", base + "!", base + ".s"]:
        if mt5.symbol_info(cand) is not None:
            mt5.symbol_select(cand, True)
            return cand
    matches = mt5.symbols_get(f"*{base}*")
    if matches:
        mt5.symbol_select(matches[0].name, True)
        return matches[0].name
    return None

def get_rates(sym: str, tf_str: str, n: int = 300) -> Optional[pd.DataFrame]:

```

```

bars = mt5.copy_rates_from_pos(sym, TF[tf_str], 0, n)
if bars is None or len(bars) < 50:
    return None
df = pd.DataFrame(bars)
df["time"] = pd.to_datetime(df["time"], unit="s")
return df

def is_crypto(sym: str) -> bool:
    s = sym.upper()
    return any(c in s for c in ["BTC", "ETH", "XRP", "LTC", "DOGE", "SOL"])

def in_killzone() -> bool:
    h = datetime.now(timezone.utc).hour
    return (LONDON_KZ[0] <= h < LONDON_KZ[1]) or (NEWYORK_KZ[0] <= h < NEWYORK_KZ[1])

def spread_points(sym: str) -> int:
    info = mt5.symbol_info(sym)
    return info.spread if info else 9999

# =====
# SMC / ICT CORE ANALYSIS
# =====

def mark_pivots(df: pd.DataFrame, lb: int) -> pd.DataFrame:
    """Mark swing highs (PH) and swing lows (PL) using N-bar fractal."""
    df = df.copy()
    df["ph"] = False
    df["pl"] = False
    for i in range(lb, len(df) - lb):
        win_h = df["high"].iloc[i - lb:i + lb + 1]
        win_l = df["low"].iloc[i - lb:i + lb + 1]
        if df["high"].iloc[i] == win_h.max() and (win_h == df["high"].iloc[i]).sum() == 1:
            df.iat[i, df.columns.get_loc("ph")] = True
        if df["low"].iloc[i] == win_l.min() and (win_l == df["low"].iloc[i]).sum() == 1:
            df.iat[i, df.columns.get_loc("pl")] = True
    return df

def htf_bias(sym: str) -> Optional[str]:
    """
    H1 market structure bias.
    Look at last 4 confirmed pivots:
        HH + HL sequence => UP
        LH + LL sequence => DOWN
    """
    df = get_rates(sym, "H1", 250)
    if df is None:
        return None
    df = mark_pivots(df, PIVOT_LOOKBACK_H1)
    phs = df[df["ph"]].tail(3)["high"].values
    pls = df[df["pl"]].tail(3)["low"].values
    if len(phs) < 2 or len(pls) < 2:
        return None
    if phs[-1] > phs[-2] and pls[-1] > pls[-2]:
        return "UP"
    if phs[-1] < phs[-2] and pls[-1] < pls[-2]:
        return "DOWN"
    return None

def detect_sweep(df_m15: pd.DataFrame, direction: str) -> Optional[dict]:
    """
    Detect liquidity sweep on M15.
    For LONG (direction=UP): wick below recent swing low + close back inside.
    For SHORT (direction=DOWN): wick above recent swing high + close back inside.
    """
    last = df_m15.iloc[-2] # last closed candle
    window = df_m15.iloc[-(SWEEP_LOOKBACK_M15 + 2):-2]
    if len(window) < 10:
        return None

```

```

    if direction == "UP":
        prior_low = window["low"].min()
        if last["low"] < prior_low and last["close"] > prior_low:
            return {"swept_level": prior_low, "sweep_low": last["low"]}
    else:
        prior_high = window["high"].max()
        if last["high"] > prior_high and last["close"] < prior_high:
            return {"swept_level": prior_high, "sweep_high": last["high"]}
    return None

def find_fvg_after_sweep(df_m5: pd.DataFrame, direction: str,
                        min_idx: int = 0) -> Optional[dict]:
    """
    Find the most recent unfilled Fair Value Gap on M5 in trade direction.
    Bullish FVG: candle[i-2].high < candle[i].low (gap between)
    Bearish FVG: candle[i-2].low > candle[i].high
    """
    fvgs = []
    for i in range(max(2, min_idx), len(df_m5)):
        c1 = df_m5.iloc[i - 2]
        c3 = df_m5.iloc[i]
        if direction == "UP" and c1["high"] < c3["low"]:
            fvgs.append({"type": "BULL", "top": c3["low"], "bot": c1["high"],
                        "mid": (c3["low"] + c1["high"]) / 2, "idx": i})
        if direction == "DOWN" and c1["low"] > c3["high"]:
            fvgs.append({"type": "BEAR", "top": c1["low"], "bot": c3["high"],
                        "mid": (c1["low"] + c3["high"]) / 2, "idx": i})

    if not fvgs:
        return None

    # Take the most recent FVG that hasn't been filled (price hasn't fully passed thru)
    latest_close = df_m5.iloc[-1]["close"]
    for fvg in reversed(fvgs):
        if direction == "UP":
            # Bullish FVG still valid if current close above the gap top (not yet retraced)
            if latest_close > fvg["top"]:
                return fvg
        else:
            if latest_close < fvg["bot"]:
                return fvg
    return None

def detect_choch_m5(df_m5: pd.DataFrame, direction: str, sweep_idx: int) -> bool:
    """
    Change of Character on M5 after the sweep:
    For UP: price closed above the most recent M5 swing high formed AFTER the sweep low.
    For DOWN: vice versa.
    """
    pdf = mark_pivots(df_m5, PIVOT_LOOKBACK_M15)
    after_sweep = pdf.iloc[sweep_idx:]
    if direction == "UP":
        recent_ph = after_sweep[after_sweep["ph"]].tail(1)
        if recent_ph.empty:
            return False
        ph_price = recent_ph["high"].iloc[0]
        return df_m5.iloc[-1]["close"] > ph_price
    else:
        recent_pl = after_sweep[after_sweep["pl"]].tail(1)
        if recent_pl.empty:
            return False
        pl_price = recent_pl["low"].iloc[0]
        return df_m5.iloc[-1]["close"] < pl_price

def find_target_liquidity(df_h1: pd.DataFrame, direction: str,
                        entry: float) -> Optional[float]:
    """Next opposing liquidity pool (next swing high for longs / low for shorts)."""
    pdf = mark_pivots(df_h1, PIVOT_LOOKBACK_H1)
    if direction == "UP":
        future_highs = pdf[pdf["ph"] & (pdf["high"] > entry)]["high"]
        if not future_highs.empty:
            return float(future_highs.min())

```

```

        return float(pdf["high"].tail(50).max())
    else:
        future_lows = pdf[pdf["pl"] & (pdf["low"] < entry)]["low"]
        if not future_lows.empty:
            return float(future_lows.max())
        return float(pdf["low"].tail(50).min())

# =====
# POSITION SIZING + ORDERS
# =====

def calc_lot(sym: str, entry: float, sl: float, risk_pct: float) -> float:
    acc = mt5.account_info()
    info = mt5.symbol_info(sym)
    if acc is None or info is None:
        return 0.0
    risk_money = acc.balance * (risk_pct / 100.0)
    sl_dist = abs(entry - sl)
    if sl_dist <= 0:
        return 0.0
    tick_size = info.trade_tick_size or info.point
    tick_value = info.trade_tick_value
    if not tick_size or not tick_value:
        return 0.0
    money_per_lot = (sl_dist / tick_size) * tick_value
    if money_per_lot <= 0:
        return 0.0
    lot = risk_money / money_per_lot
    step = info.volume_step or 0.01
    lot = round(lot / step) * step
    lot = max(info.volume_min, min(info.volume_max, lot))
    return round(lot, 2)

def place_pending_limit(sym: str, side: str, lot: float, price: float,
                        sl: float, tp: float, comment: str) -> Optional[int]:
    """Pending limit at FVG midpoint - fills only if price retraces."""
    info = mt5.symbol_info(sym)
    if info is None or lot <= 0:
        return None
    digits = info.digits
    otype = mt5.ORDER_TYPE_BUY_LIMIT if side == "BUY" else mt5.ORDER_TYPE_SELL_LIMIT
    expiry = int((datetime.now() + timedelta(hours=PENDING_ORDER_EXPIRY_HOURS)).timestamp())
    req = {
        "action": mt5.TRADE_ACTION_PENDING,
        "symbol": sym, "volume": lot, "type": otype,
        "price": round(price, digits),
        "sl": round(sl, digits),
        "tp": round(tp, digits),
        "deviation": 30,
        "magic": MAGIC_NUMBER,
        "comment": comment[:31],
        "type_time": mt5.ORDER_TIME_SPECIFIED,
        "expiration": expiry,
        "type_filling": mt5.ORDER_FILLING_RETURN,
    }
    r = mt5.order_send(req)
    if r is None or r.retcode != mt5.TRADE_RETCODE_DONE:
        msg = getattr(r, "comment", "") if r else mt5.last_error()
        print(f"[!] {sym} {side} LIMIT failed: {msg}")
        return None
    print(f"[+] {sym} {side}_LIMIT {lot} @ {price:.5f} "
          f"SL={sl:.5f} TP={tp:.5f} ({comment})")
    return r.order

def modify_sl(pos, new_sl: float) -> bool:
    info = mt5.symbol_info(pos.symbol)
    digits = info.digits if info else 5
    req = {
        "action": mt5.TRADE_ACTION_SLTP,
        "position": pos.ticket,
        "symbol": pos.symbol,
        "sl": round(new_sl, digits),
    }

```

```

        "tp": round(pos.tp, digits),
    }
    r = mt5.order_send(req)
    return r is not None and r.retcode == mt5.TRADE_RETCODE_DONE

# ----- State persistence -----
def load_state() -> dict:
    if os.path.exists(STATE_FILE):
        try:
            with open(STATE_FILE) as f:
                return json.load(f)
        except Exception:
            pass
    return {}

def save_state(s: dict):
    try:
        with open(STATE_FILE, "w") as f:
            json.dump(s, f)
    except Exception as e:
        print(f"[!] state save failed: {e}")

def open_count(sym: Optional[str] = None) -> int:
    pos = mt5.positions_get(symbol=sym) if sym else mt5.positions_get()
    if pos is None:
        return 0
    return sum(1 for p in pos if p.magic == MAGIC_NUMBER)

def pending_count(sym: Optional[str] = None) -> int:
    orders = mt5.orders_get(symbol=sym) if sym else mt5.orders_get()
    if orders is None:
        return 0
    return sum(1 for o in orders if o.magic == MAGIC_NUMBER)

# =====
# ENTRY PIPELINE - SMC/ICT confluence check
# =====

def try_smc_setup(sym: str, state: dict) -> bool:
    # Caps
    if open_count() + pending_count() >= MAX_OPEN_TRADES:
        return False
    if open_count(sym) + pending_count(sym) >= MAX_TRADES_PER_PAIR:
        return False

    # Killzone filter - only forex; crypto 24/7
    if not is_crypto(sym) and not in_killzone():
        return False

    # Spread filter
    cap = MAX_SPREAD_CRYPTO if is_crypto(sym) else MAX_SPREAD_FX
    if spread_points(sym) > cap:
        return False

    # 1. HTF bias (H1)
    direction = htf_bias(sym)
    if direction is None:
        return False

    # 2. Liquidity sweep on M15
    df_m15 = get_rates(sym, "M15", 200)
    if df_m15 is None:
        return False
    sweep = detect_sweep(df_m15, direction)
    if sweep is None:
        return False

    # 3. CHoCH on M5
    df_m5 = get_rates(sym, "M5", 200)
    if df_m5 is None:

```

```

        return False
    # Approximate sweep index on M5: last 1-2 hours of M5 = ~24 candles
    sweep_idx_m5 = max(0, len(df_m5) - 30)
    if not detect_choch_m5(df_m5, direction, sweep_idx_m5):
        return False

    # 4. Find unfilled FVG on M5 in trade direction (POI for entry)
    fvg = find_fvg_after_sweep(df_m5, direction, min_idx=sweep_idx_m5)
    if fvg is None:
        return False

    # 5. Build entry/SL/TP
    atr_m15 = atr(df_m15, ATR_PERIOD).iloc[-1]
    if pd.isna(atr_m15):
        return False

    entry = fvg["mid"]                                # enter at FVG midpoint via limit
    if direction == "UP":
        sl = sweep["sweep_low"] - SL_BUFFER_ATR * atr_m15
    else:
        sl = sweep["sweep_high"] + SL_BUFFER_ATR * atr_m15

    risk_dist = abs(entry - sl)
    if risk_dist <= 0:
        return False

    # 6. Find target liquidity
    df_h1 = get_rates(sym, "H1", 200)
    target = find_target_liquidity(df_h1, direction, entry) if df_h1 is not None else None
    if target is None:
        # fallback: 2R
        target = entry + 2 * risk_dist if direction == "UP" else entry - 2 * risk_dist

    rr = abs(target - entry) / risk_dist
    if rr < MIN_RR_TO_TARGET:
        print(f"[~] {sym} setup found but RR={rr:.2f} too low; skip")
        return False

    # 7. Place pending limit at FVG midpoint
    side = "BUY" if direction == "UP" else "SELL"
    lot = calc_lot(sym, entry, sl, RISK_PERCENT)
    if lot <= 0:
        print(f"[!] {sym} lot=0 - check tick value")
        return False

    print(f"[*] SMC setup: {sym} {side} | bias={direction} | sweep@{sweep['swept_level']:.5f} "
          f"| FVG entry={entry:.5f} | SL={sl:.5f} | TP={target:.5f} | RR={rr:.2f}")

    ticket = place_pending_limit(sym, side, lot, entry, sl, target,
                                f"SMC_{direction}_RR{rr:.1f}")

    if ticket is None:
        return False

    state[str(ticket)] = {
        "type": "pending",
        "symbol": sym, "side": side,
        "entry": entry, "initial_sl": sl, "tp": target,
        "r_distance": risk_dist, "stage": 0,
    }
    save_state(state)
    return True

def scan_all(state: dict, symbols: list) -> int:
    placed = 0
    for sym in symbols:
        try:
            if try_smc_setup(sym, state):
                placed += 1
        except Exception as e:
            print(f"[!] {sym} scan error: {e}")
    return placed

# =====

```

```

# TRADE MANAGEMENT - promote pending->position, then smart trail
# =====

def manage_trades(state: dict):
    # Promote: when pending fills, MT5 creates a position with same ticket-or-deal
    positions = mt5.positions_get() or []

    # Build lookup of currently active tickets
    pos_tickets = {str(p.ticket): p for p in positions if p.magic == MAGIC_NUMBER}

    # Move state entries from "pending" -> "position" for filled orders
    for tk, st in list(state.items()):
        if st.get("type") == "pending" and tk in pos_tickets:
            st["type"] = "position"
            print(f"[+] Pending #{tk} filled - now active position")
            save_state(state)

    # Trail open positions
    for pos in positions:
        if pos.magic != MAGIC_NUMBER:
            continue
        st = state.get(str(pos.ticket))
        if st is None:
            continue
        rd = st.get("r_distance", 0)
        if rd <= 0:
            continue

        tick = mt5.symbol_info_tick(pos.symbol)
        if tick is None:
            continue
        cur = tick.bid if pos.type == mt5.ORDER_TYPE_BUY else tick.ask
        entry = st["entry"]

        if pos.type == mt5.ORDER_TYPE_BUY:
            profit_r = (cur - entry) / rd
        else:
            profit_r = (entry - cur) / rd

        if profit_r <= 0:
            continue

        new_sl = pos.sl
        stage = st.get("stage", 0)

        # +1R -> BE
        if profit_r >= BREAKEVEN_AT_R and stage < 1:
            new_sl = entry
            st["stage"] = 1
            print(f"[~] {pos.symbol} #{pos.ticket} -> BE @ {profit_r:.2f}R")

        # +1.5R -> lock half
        if profit_r >= LOCK_HALF_AT_R and stage < 2:
            new_sl = entry + 0.5 * rd if pos.type == mt5.ORDER_TYPE_BUY else entry - 0.5 * rd
            st["stage"] = 2
            print(f"[~] {pos.symbol} #{pos.ticket} -> LOCK +0.5R @ {profit_r:.2f}R")

        # +2R+ -> ATR Chandelier trail
        if profit_r >= TRAIL_START_R:
            df = get_rates(pos.symbol, "M15", 50)
            if df is not None:
                a = atr(df, ATR_PERIOD).iloc[-1]
                if pos.type == mt5.ORDER_TYPE_BUY:
                    chand = cur - ATR_TRAIL_MULT * a
                    if chand > new_sl:
                        new_sl = chand
                else:
                    chand = cur + ATR_TRAIL_MULT * a
                    if new_sl == 0 or chand < new_sl:
                        new_sl = chand
            st["stage"] = max(stage, 3)

    improve = (
        (pos.type == mt5.ORDER_TYPE_BUY and new_sl > pos.sl)
        or (pos.type == mt5.ORDER_TYPE_SELL and (pos.sl == 0 or new_sl < pos.sl))
    )

```

```

    )
    if improve and modify_sl(pos, new_sl):
        print(f"    SL: {pos.sl:.5f} -> {new_sl:.5f} (R={profit_r:.2f})")

# Cleanup: drop state for tickets that are neither open positions nor pending orders
pending_tickets = {str(o.ticket) for o in (mt5.orders_get() or [])
                    if o.magic == MAGIC_NUMBER}
alive = set(pos_tickets.keys()) | pending_tickets
for tk in list(state.keys()):
    if tk not in alive:
        state.pop(tk, None)
save_state(state)

# =====
#  MAIN
# =====
def main():
    if not init_mt5():
        return

    requested = SYMBOLS_FOREX + SYMBOLS_CRYPTO
    symbols = []
    for s in requested:
        actual = find_symbol(s)
        if actual:
            symbols.append(actual)
            print(f"    [+] {s} -> {actual}")
        else:
            print(f"    [-] {s} not available")

    if not symbols:
        print("[!] No symbols.")
        mt5.shutdown()
        return

    state = load_state()
    end = (datetime.now() + timedelta(minutes=RUN_MINUTES)) if RUN_MINUTES > 0 else None
    print(f"\n[*] SMC/ICT Bot live. Risk={RISK_PERCENT}% | Symbols={len(symbols)} | "
          f"'Forever' if end is None else f'Till {end:%H:%M:%S}')"
    print(f"[*] Killzones (UTC): London {LONDON_KZ[0]:02d}-{LONDON_KZ[1]:02d}, "
          f"NY {NEWYORK_KZ[0]:02d}-{NEWYORK_KZ[1]:02d} | "
          f"Now: {datetime.now(timezone.utc):%H:%M} UTC | "
          f"In KZ: {in_killzone()}\n")

    try:
        while True:
            if end and datetime.now() >= end:
                print("[*] Time up.")
                break
            placed = scan_all(state, symbols)
            manage_trades(state)
            if placed:
                print(f"[i] Placed {placed} | Open: {open_count()} | "
                      f"Pending: {pending_count()}")
                time.sleep(SCAN_INTERVAL_SEC)
    except KeyboardInterrupt:
        print("\n[*] Stopped by user.")
    finally:
        mt5.shutdown()
        print("[*] Disconnected.")

if __name__ == "__main__":
    main()

```

Path in repo: /mt5_final_bot.py

[illegible]

```

# Initial Risk:Reward target (fallback TP if trailing fails)
INITIAL_RR = 2.0

# Trailing thresholds (in R-multiples)
BREAKEVEN_AT_R = 1.0      # Move SL to entry when profit reaches 1R
LOCK_HALF_AT_R = 1.5      # Move SL to +0.5R when profit reaches 1.5R
TRAIL_START_R = 2.0       # Start ATR-trailing after this
ATR_TRAIL_MULT = 1.5      # Chandelier exit distance

# Trade caps - take every valid setup
MAX_OPEN_TRADES = 999      # effectively unlimited
MAX_TRADES_PER_PAIR = 1    # still 1 per symbol

# Higher TF trend filter
HTF_EMA_FAST = 50
HTF_EMA_SLOW = 200
ADX_MIN = 22

# Entry filters
ENTRY_EMA = 20             # M15 pullback EMA
RSI_PERIOD = 14
ATR_PERIOD = 14
SL_ATR_MULT = 1.5

# Loop
SCAN_INTERVAL_SEC = 5
RUN_MINUTES = 0           # 0 = forever
MAGIC_NUMBER = 777001

# Spread cap (in points). Skip if broker spread > this.
MAX_SPREAD_POINTS_FX = 25
MAX_SPREAD_POINTS_CRYPTO = 5000 # crypto has wider spreads

# State persistence (for trailing - survives restart)
STATE_FILE = "mt5_bot_state.json"

# =====

# ----- Indicators -----
def ema(s: pd.Series, p: int) -> pd.Series:
    return s.ewm(span=p, adjust=False).mean()

def rsi(s: pd.Series, p: int = 14) -> pd.Series:
    d = s.diff()
    g = d.clip(lower=0).ewm(alpha=1/p, adjust=False).mean()
    l = (-d.clip(upper=0)).ewm(alpha=1/p, adjust=False).mean()
    rs = g / l.replace(0, np.nan)
    return 100 - (100 / (1 + rs))

def atr(df: pd.DataFrame, p: int = 14) -> pd.Series:
    h, l, c = df["high"], df["low"], df["close"]
    tr = pd.concat([h - l, (h - c.shift()).abs(), (l - c.shift()).abs()], axis=1).max(axis=1)
    return tr.ewm(alpha=1/p, adjust=False).mean()

def adx(df: pd.DataFrame, p: int = 14) -> pd.Series:
    h, l, c = df["high"], df["low"], df["close"]
    plus_dm = (h.diff()).where(lambda x: x > 0, 0)
    minus_dm = (-l.diff()).where(lambda x: x > 0, 0)
    plus_dm = plus_dm.where(plus_dm > minus_dm, 0)
    minus_dm = minus_dm.where(minus_dm > plus_dm, 0)
    tr = pd.concat([h - l, (h - c.shift()).abs(), (l - c.shift()).abs()], axis=1).max(axis=1)
    atr_v = tr.ewm(alpha=1/p, adjust=False).mean()
    plus_di = 100 * plus_dm.ewm(alpha=1/p, adjust=False).mean() / atr_v.replace(0, np.nan)
    minus_di = 100 * minus_dm.ewm(alpha=1/p, adjust=False).mean() / atr_v.replace(0, np.nan)
    dx = 100 * (plus_di - minus_di).abs() / (plus_di + minus_di).replace(0, np.nan)
    return dx.ewm(alpha=1/p, adjust=False).mean()

# ----- MT5 helpers -----
TF = {"M5": mt5.TIMEFRAME_M5, "M15": mt5.TIMEFRAME_M15,

```

```

    "H1": mt5.TIMEFRAME_H1, "H4": mt5.TIMEFRAME_H4}

def init_mt5() -> bool:
    kw = {"login": MT5_LOGIN, "password": MT5_PASSWORD, "server": MT5_SERVER}
    if MT5_PATH:
        kw["path"] = MT5_PATH
    if not mt5.initialize(**kw):
        print(f"[!] MT5 init failed: {mt5.last_error()}")
        return False
    acc = mt5.account_info()
    if acc is None:
        print(f"[!] Login failed: {mt5.last_error()}")
        return False
    print(f"[+] Logged in: {acc.login} | Balance: {acc.balance:.2f} {acc.currency} | Server: {acc.server}")
    return True

def find_symbol(base: str) -> Optional[str]:
    """Auto-detect broker-specific symbol name (BTCUSD, BTCUSDM, BTCUSD.r, etc.)"""
    for cand in [base, base + "m", base + ".r", base + "_", base + "!", base + ".s"]:
        info = mt5.symbol_info(cand)
        if info is not None:
            mt5.symbol_select(cand, True)
            return cand
    matches = mt5.symbols_get(f"*{base}*")
    if matches:
        name = matches[0].name
        mt5.symbol_select(name, True)
        return name
    return None

def get_rates(sym: str, tf_str: str, n: int = 300) -> Optional[pd.DataFrame]:
    bars = mt5.copy_rates_from_pos(sym, TF[tf_str], 0, n)
    if bars is None or len(bars) < 50:
        return None
    df = pd.DataFrame(bars)
    df["time"] = pd.to_datetime(df["time"], unit="s")
    return df

def spread_points(sym: str) -> int:
    info = mt5.symbol_info(sym)
    return info.spread if info else 9999

def is_crypto(sym: str) -> bool:
    s = sym.upper()
    return any(c in s for c in ["BTC", "ETH", "XRP", "LTC", "BCH", "DOGE", "SOL"])

# ----- Multi-timeframe analysis -----
def htf_trend(sym: str) -> Optional[str]:
    """H4 macro trend: 'UP' / 'DOWN' / None"""
    df = get_rates(sym, "H4", 250)
    if df is None:
        return None
    df["ef"] = ema(df["close"], HTF_EMA_FAST)
    df["es"] = ema(df["close"], HTF_EMA_SLOW)
    last = df.iloc[-1]
    if last["close"] > last["ef"] > last["es"]:
        return "UP"
    if last["close"] < last["ef"] < last["es"]:
        return "DOWN"
    return None

def h1_strength(sym: str, direction: str) -> bool:
    """H1 confirms trend strength via ADX + EMA50 alignment"""
    df = get_rates(sym, "H1", 200)
    if df is None:
        return False
    df["ema50"] = ema(df["close"], 50)

```

```

    df["adx"] = adx(df, 14)
    last = df.iloc[-1]
    if pd.isna(last["adx"]) or last["adx"] < ADX_MIN:
        return False
    if direction == "UP":
        return last["close"] > last["ema50"]
    return last["close"] < last["ema50"]

def m15_entry(sym: str, direction: str):
    """M15 pullback + RSI cross. Returns (entry_price, sl_price, atr_value) or None"""
    df = get_rates(sym, "M15", 200)
    if df is None:
        return None
    df["e20"] = ema(df["close"], ENTRY_EMA)
    df["rsi"] = rsi(df["close"], RSI_PERIOD)
    df["atr"] = atr(df, ATR_PERIOD)
    a, b = df.iloc[-1], df.iloc[-2] # last closed, prev closed
    if pd.isna(a["atr"]) or pd.isna(a["rsi"]):
        return None

    atr_val = a["atr"]
    price = a["close"]

    if direction == "UP":
        # Pullback near EMA20, RSI crossed up through 50, bullish candle
        if (b["low"] <= b["e20"] * 1.001 # touched/near EMA20
            and b["rsi"] < 50 <= a["rsi"] # RSI crossed up
            and a["close"] > a["open"]): # bullish close
            sl = price - SL_ATR_MULT * atr_val
            return (price, sl, atr_val)

    if direction == "DOWN":
        if (b["high"] >= b["e20"] * 0.999
            and b["rsi"] > 50 >= a["rsi"]
            and a["close"] < a["open"]):
            sl = price + SL_ATR_MULT * atr_val
            return (price, sl, atr_val)

    return None

def m5_trigger(sym: str, direction: str) -> bool:
    """M5 confirmation: candle in trend direction with momentum"""
    df = get_rates(sym, "M5", 50)
    if df is None:
        return False
    a = df.iloc[-2]
    body = abs(a["close"] - a["open"])
    rng = a["high"] - a["low"]
    if rng <= 0:
        return False
    if body / rng < 0.5: # require strong body (>50% of range)
        return False
    if direction == "UP":
        return a["close"] > a["open"]
    return a["close"] < a["open"]

# ----- Position sizing -----
def calc_lot(sym: str, entry: float, sl: float, risk_pct: float) -> float:
    """Compute lot size so that loss at SL == risk_pct of balance."""
    acc = mt5.account_info()
    info = mt5.symbol_info(sym)
    if acc is None or info is None:
        return 0.0

    risk_money = acc.balance * (risk_pct / 100.0)
    sl_dist = abs(entry - sl)
    if sl_dist <= 0:
        return 0.0

    tick_size = info.trade_tick_size or info.point
    tick_value = info.trade_tick_value
    if not tick_size or not tick_value:

```

```

        return 0.0

    money_per_lot = (sl_dist / tick_size) * tick_value
    if money_per_lot <= 0:
        return 0.0

    lot = risk_money / money_per_lot
    step = info.volume_step or 0.01
    lot = round(lot / step) * step
    lot = max(info.volume_min, min(info.volume_max, lot))
    return round(lot, 2)

# ----- Order placement -----
def place_order(sym: str, side: str, lot: float, sl: float, tp: float,
                comment: str) -> Optional[int]:
    info = mt5.symbol_info(sym)
    tick = mt5.symbol_info_tick(sym)
    if info is None or tick is None or lot <= 0:
        return None

    digits = info.digits
    if side == "BUY":
        price, otype = tick.ask, mt5.ORDER_TYPE_BUY
    else:
        price, otype = tick.bid, mt5.ORDER_TYPE_SELL

    req = {
        "action": mt5.TRADE_ACTION_DEAL,
        "symbol": sym, "volume": lot, "type": otype,
        "price": round(price, digits),
        "sl": round(sl, digits),
        "tp": round(tp, digits),
        "deviation": 30,
        "magic": MAGIC_NUMBER,
        "comment": comment[:31],
        "type_time": mt5.ORDER_TIME_GTC,
        "type_filling": mt5.ORDER_FILLING_IOC,
    }
    r = mt5.order_send(req)
    if r is None or r.retcode != mt5.TRADE_RETCODE_DONE:
        msg = getattr(r, "comment", "") if r else mt5.last_error()
        print(f"[!] {sym} {side} failed: {msg}")
        return None
    print(f"[+] {sym} {side} {lot} @ {price:.5f} SL={sl:.5f} TP={tp:.5f} ({comment})")
    return r.order

def modify_sl(pos, new_sl: float) -> bool:
    info = mt5.symbol_info(pos.symbol)
    digits = info.digits if info else 5
    req = {
        "action": mt5.TRADE_ACTION_SLTP,
        "position": pos.ticket,
        "symbol": pos.symbol,
        "sl": round(new_sl, digits),
        "tp": round(pos.tp, digits),
    }
    r = mt5.order_send(req)
    return r is not None and r.retcode == mt5.TRADE_RETCODE_DONE

# ----- Trade state (initial R distance for trailing math) -----
def load_state() -> dict:
    if os.path.exists(STATE_FILE):
        try:
            with open(STATE_FILE) as f:
                return json.load(f)
        except Exception:
            pass
    return {}

def save_state(s: dict):
    try:
        with open(STATE_FILE, "w") as f:

```

```

        json.dump(s, f)
    except Exception as e:
        print(f"[!] state save failed: {e}")

# ----- Open trades count -----
def open_count(sym: Optional[str] = None) -> int:
    pos = mt5.positions_get(symbol=sym) if sym else mt5.positions_get()
    if pos is None:
        return 0
    return sum(1 for p in pos if p.magic == MAGIC_NUMBER)

# ----- Strategy entry pipeline -----
def try_entry(sym: str, state: dict) -> bool:
    if open_count() >= MAX_OPEN_TRADES:
        return False
    if open_count(sym) >= MAX_TRADES_PER_PAIR:
        return False

    sp_cap = MAX_SPREAD_POINTS_CRYPTO if is_crypto(sym) else MAX_SPREAD_POINTS_FX
    if spread_points(sym) > sp_cap:
        return False

    # Tier 1: H4 trend
    direction = htf_trend(sym)
    if direction is None:
        return False
    # Tier 2: H1 strength
    if not h1_strength(sym, direction):
        return False
    # Tier 3: M15 entry zone
    res = m15_entry(sym, direction)
    if res is None:
        return False
    entry, sl, atr_val = res
    # Tier 4: M5 momentum trigger
    if not m5_trigger(sym, direction):
        return False

    side = "BUY" if direction == "UP" else "SELL"
    rr_dist = abs(entry - sl)
    tp = entry + INITIAL_RR * rr_dist if side == "BUY" else entry - INITIAL_RR * rr_dist

    lot = calc_lot(sym, entry, sl, RISK_PERCENT)
    if lot <= 0:
        print(f"[!] {sym} lot calc returned 0 - check tick_value")
        return False

    ticket = place_order(sym, side, lot, sl, tp, f"MTF_{direction}")
    if ticket is None:
        return False

    # Save initial R distance + ATR for trailing
    state[str(ticket)] = {
        "symbol": sym,
        "side": side,
        "entry": entry,
        "initial_sl": sl,
        "r_distance": rr_dist,
        "atr": atr_val,
        "stage": 0,
    }
    # 0 = initial, 1 = BE, 2 = locked, 3 = trailing
    save_state(state)
    return True

def scan_all(state: dict, symbols: list) -> int:
    placed = 0
    for sym in symbols:
        try:
            if try_entry(sym, state):
                placed += 1
        except Exception as e:
            print(f"[!] {sym} scan error: {e}")

```

```
return placed
```

```
# ----- Trade management (smart trailing) -----
def manage_trades(state: dict):
    positions = mt5.positions_get()
    if not positions:
        return

    for pos in positions:
        if pos.magic != MAGIC_NUMBER:
            continue
        st = state.get(str(pos.ticket))
        if st is None:
            continue

        tick = mt5.symbol_info_tick(pos.symbol)
        if tick is None:
            continue

        cur = tick.bid if pos.type == mt5.ORDER_TYPE_BUY else tick.ask
        entry = st["entry"]
        rd = st["r_distance"]      # 1R distance in price
        if rd <= 0:
            continue

        # Current profit in R-multiples
        if pos.type == mt5.ORDER_TYPE_BUY:
            profit_r = (cur - entry) / rd
        else:
            profit_r = (entry - cur) / rd

        if profit_r <= 0:
            continue      # nothing to trail in loss

        new_sl = pos.sl
        stage = st.get("stage", 0)

        # Stage 1: BE at +1R
        if profit_r >= BREAKEVEN_AT_R and stage < 1:
            new_sl = entry
            st["stage"] = 1
            print(f"[~] {pos.symbol} #{pos.ticket} -> BREAKEVEN @ {profit_r:.2f}R")

        # Stage 2: lock half at +1.5R
        if profit_r >= LOCK_HALF_AT_R and stage < 2:
            new_sl = entry + 0.5 * rd if pos.type == mt5.ORDER_TYPE_BUY else entry - 0.5 * rd
            st["stage"] = 2
            print(f"[~] {pos.symbol} #{pos.ticket} -> LOCK +0.5R @ {profit_r:.2f}R")

        # Stage 3: ATR trailing after +2R (Chandelier-style)
        if profit_r >= TRAIL_START_R:
            df = get_rates(pos.symbol, "M15", 50)
            if df is not None:
                atr_now = atr(df, ATR_PERIOD).iloc[-1]
                if pos.type == mt5.ORDER_TYPE_BUY:
                    chandelier = cur - ATR_TRAIL_MULT * atr_now
                    if chandelier > new_sl:
                        new_sl = chandelier
                else:
                    chandelier = cur + ATR_TRAIL_MULT * atr_now
                    if chandelier < new_sl or new_sl == 0:
                        new_sl = chandelier
            st["stage"] = max(stage, 3)

        # Apply only if it improves the SL (never worsen)
        improve = (
            (pos.type == mt5.ORDER_TYPE_BUY and new_sl > pos.sl)
            or (pos.type == mt5.ORDER_TYPE_SELL and (pos.sl == 0 or new_sl < pos.sl))
        )
        if improve and modify_sl(pos, new_sl):
            print(f"    SL moved: {pos.sl:.5f} -> {new_sl:.5f} (R={profit_r:.2f})")

# Cleanup state for closed positions
open_tickets = {str(p.ticket) for p in positions if p.magic == MAGIC_NUMBER}
```

```

    for tk in list(state.keys()):
        if tk not in open_tickets:
            state.pop(tk, None)
    save_state(state)

# ----- Main loop -----
def main():
    if not init_mt5():
        return

    # Resolve symbols (handles broker suffixes)
    requested = SYMBOLS_FOREX + SYMBOLS_CRYPTO
    symbols = []
    for s in requested:
        actual = find_symbol(s)
        if actual:
            symbols.append(actual)
            print(f"  [+] {s} -> {actual}")
        else:
            print(f"  [-] {s} not available on this broker")

    if not symbols:
        print("[!] No tradable symbols found.")
        mt5.shutdown()
        return

    state = load_state()
    end = (datetime.now() + timedelta(minutes=RUN_MINUTES)) if RUN_MINUTES > 0 else None
    print(f"\n[*] Running. Risk={RISK_PERCENT}% per trade | Symbols={len(symbols)} | "
          f"'Forever' if end is None else f'Till {end:%H:%M:%S}'")

    try:
        while True:
            if end and datetime.now() >= end:
                print("[*] Time up.")
                break
            placed = scan_all(state, symbols)
            manage_trades(state)
            if placed:
                print(f"[i] Placed {placed} | Open: {open_count()}")
                time.sleep(SCAN_INTERVAL_SEC)
    except KeyboardInterrupt:
        print("\n[*] Stopped by user.")
    finally:
        mt5.shutdown()
        print("[*] MT5 disconnected. Open trades remain managed by SL/TP on broker.")

if __name__ == "__main__":
    main()

```